

Objective

To implement and demonstrate `IllegalStateException` in Java using try-catch by attempting to use a Scanner after it has been closed.

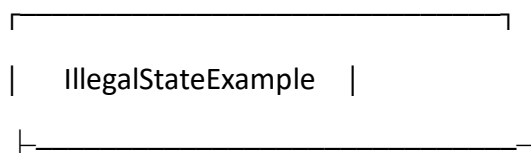
Steps for Execution

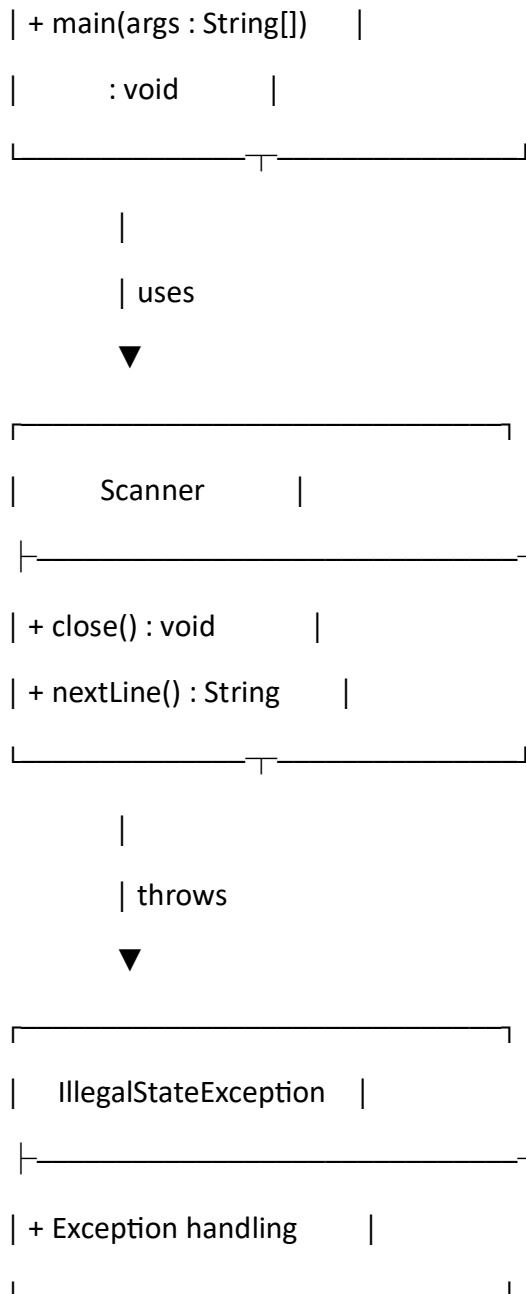
1. Create a Java project and package named `javacore`.
2. Create a file named `IllegalStateExceptionExample.java` and enter the given code.
3. Create a Scanner object.
4. Close the scanner.
5. Try to read input using the closed scanner.
6. Handle the exception using catch.
7. Compile and run the program.
8. Observe the output.

Algorithm

1. Start the program.
2. Create a Scanner object.
3. Close the scanner.
4. Try to read input using `nextLine()`.
5. Catch `IllegalStateException`.
6. Display the error message.
7. Display "Program ended."
8. Stop the program.

UML Class Diagram





Relationship

- `IllegalStateException` creates and uses a `Scanner` object.
- The scanner is closed using `close()`.
- Calling `nextLine()` after closing the scanner causes an `IllegalStateException`.
- The exception is handled using the catch block.

CODE:

```
package javacore;

import java.util.Scanner;

public class IllegalStateExceptionExample {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        try {

            System.out.println("Scanner is ready.");

            scanner.close();

            System.out.println("Scanner is closed.");

            scanner.nextLine();

        }

        catch (IllegalStateException e) {

            System.out.println("Error: Scanner is already closed.");

        }

        System.out.println("Program ended.");

    }

}
```

Output

Scanner is ready.

Scanner is closed.

Error: Scanner is already closed.

Program ended.

Result

The program successfully demonstrates `IllegalStateException` handling in Java. Since `nextLine()` is called after the Scanner has been closed, an `IllegalStateException` occurs and is handled using the catch block.